STELF as a Logic

As anyone who has programmed in Prolog will know, there are a few problems with Prolog to formalize logic

Need cite

. Among all of these, the problems can be largly divided into four groups.

1. Apart from system errors, there is no notion of "failure" and seperate from "falsehood". Per as a matter of fact, the Prolog term `fail` is just `false`

   Cite SWI docs

   . This can lead to confusing bugs, where a statement is said to be false due to being poorly formed, in addition, type checks have their own issues with initilization
2. "Metavariables" as it turns out are rather vague a concept. Particular, the ambiguity of this (not even including CLP) requires three different equality predicates, = for "simple" equality, == for "trivial" equality, and =@= for "trivial up to α-renaming"
3. Adding imperitive features to a backtracing language introduces errors
4. Cut, which seeks to solve all these problems, due to Prolog's non-monotonic nature makes a number of them *worse*.

All of these together make Prolog, which is a useful programming language, unsuitable for formalizing logic. A number of solutions to these problems, particularly the issue of ensuring that terms are well formed, have been presented. Of note, are λProlog and Mecury, which add types to Prolog. However, each of this is complex, and bears some of the issues with Prolog still (in particular, λProlog still has cut).

These problems are solved simply and elegantly in the Endibrugh Logical Framework, which we will first look at as a logic.

Paeno Arithmetic

Let us take the example of Paeno arithemetic on the natural numbers.

We first define a reference implementation in Prolog:

```
:- public nat/1.
nat(zero).
nat(succ(N)) :- nat(N).
:- public add/3.
add(X, zero, X).
add(X, succ(Y), succ(Z)) :- add(X, Y, Z).
:- public mul/3.
mul(X, zero, zero).
mul(X, succ(Y), Z) :- mul(X, Y, Z1), add(Z1, X, Z).
```

The first two lines define the natural numbers, as an inductive predicate, the next two define addition, and the last two define multiplication.

We start off with decleration of the sort of natural numbers, which isn't found in the above Prolog code

A Primer in Greek

Definition of Canonicity

Consider two syntactic categories, A and B. Let A_ρ be A equipped with the reflexive, transitive, symmetric closure of the reduction relation $\underset{\rho}{\rightarrow}$ (such that there is a functor $\text{equip}_\rho : A \Rightarrow A_\rho$). Show that for each A_ρ connected component, there exists a unique B